Swift on Android

Building an idiomatic Cross-Platform Library with Swift/Java Interop

Mads Odgaard, Swift @ FOSDEM 2026

Who am I?

- Tech Lead @ Frameo
- Worked on swift-java in GSoC 2025
- Swift Enthusiast
- Interested in cryptography, networking and programming languages

🇩🇰

Mads Odgaard

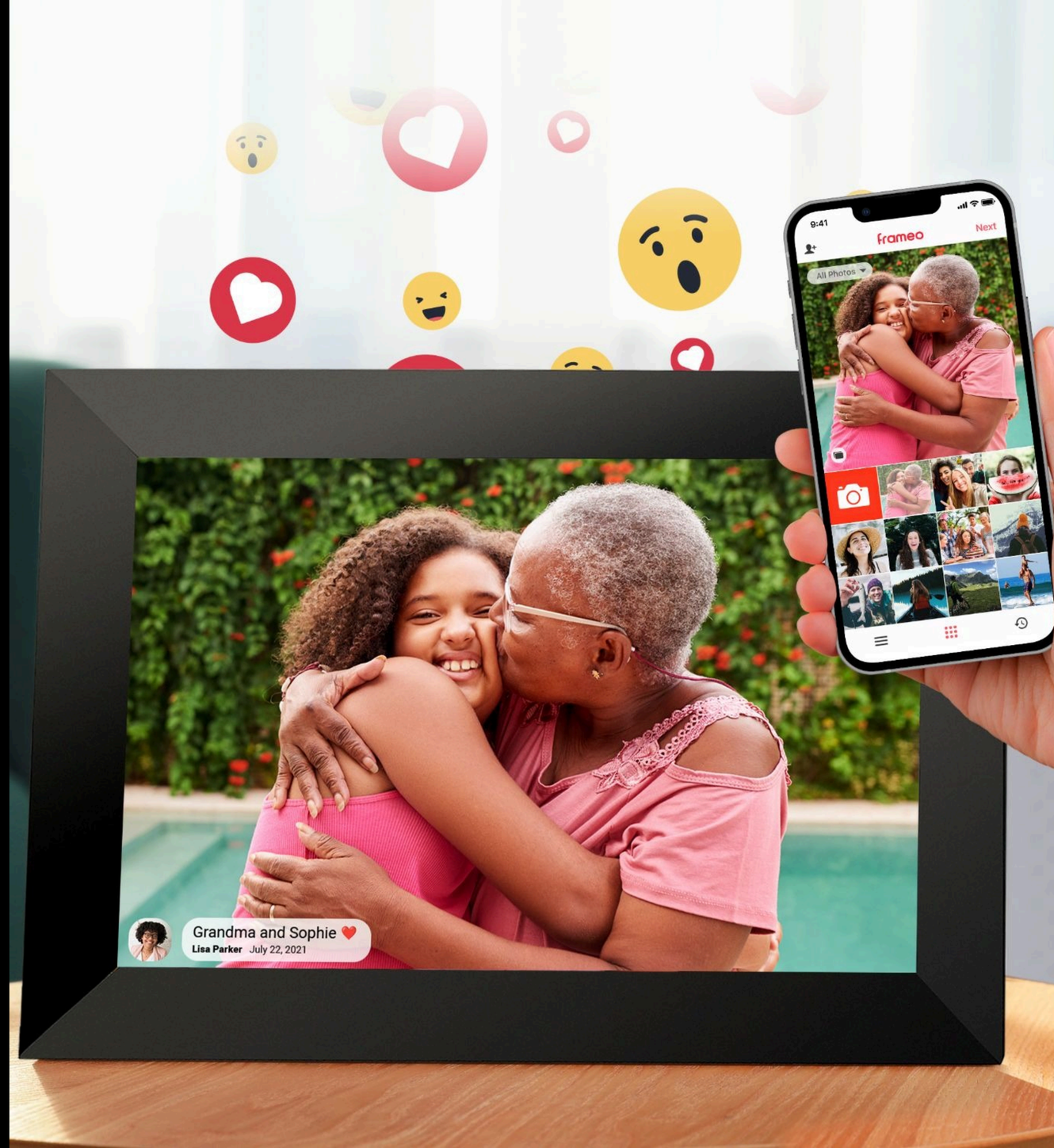
mads@madsodgaard.com

@mads-odgaard

@madsodgaard

frameo

- Digital Photo Frame Software
- 3 platforms
  - iOS app
  - Android app
  - Frame Software (Android)



Frameo in numbers

20M+ downloads

2.2B photos sent

17.5M Android devices

17.5M Android devices

New Feature

Not in production yet

“Chia” library

- Swift library to handle media transmissions between Client and Frame
- Implements Signal-like protocol.
  - Handles encryption, networking, storage and more.
- Used on both iOS, Android client and Frame.

Package.swift

swift-chia

Chia

DoubleRatchet

MessageServerClient

MediaServerClient

Protobuf

swift-libsodium

swift-openapi-generator

swift-protobuf

swift-async-http-client

swift-async-algorithms

swift-logging

swift-collections

swift-nio

Interacting with the library

```
public final class ChiaClient: Sendable {
    /// Encrypts and sends a message to the recipients
    public func send(encrypting message: ChiaMessageType, to recipientIDs: [ChiaID]) async throws

    /// Requests a pre-signed URL to upload media
    public func requestMediaUpload(recipients: [ChiaID]) async throws -> MediaUpload

    /// Encrypts a file on disk and saves it to a new location
    public func encryptMedia(storedAt inputFileURL: URL, savingTo outputURL: URL) async throws -> MediaEncryption

   /// Downloads a media from the server, decrypts it and saves it to the disk.
    public func downloadAndDecryptMedia(
        _ mediaMetadata: EncryptedMediaMetadata,
        streamingToPath outputFilePath: String
    ) async throws
}
```

Initialization

```swift
public final class ChiaClient: Sendable {

    // ...

    public convenience init(
        deviceID: ChiaID,
        privateKey: Curve25519.PrivateKey,
        storage: any ChiaStorage,
        logger: Logger,
        messageHandler: any ChiaMessageHandler
    )

    // ...
```

```swift
public protocol ChiaStorage: Sendable {
    func saveSession(_ session: ChiaSession) throws

    func getActiveSession(with id: ChiaID) throws -> ChiaSession?
    func setActiveSession(with id: ChiaID, sessionID: String) throws
    func getSessions(with id: ChiaID) throws -> [ChiaSession]

    func getDevices() throws -> [ChiaDevice]
    func getDevice(withID ids: ChiaID) throws -> ChiaDevice?
    func saveDevice(_ device: ChiaDevice) throws
    func deleteDevices(_ ids: [ChiaID]) throws
}
```

```swift
public protocol ChiaMessageHandler {
    func messageReceived(_ message: ChiaMessageType)
}
```

Now let’s do it all again!

Just in Java…

SwiftJava

https://github.com/swiftlang/swift-java

Swift Java: JExtract

- Java source-generation tool
- Call Swift from Java
- JNI mode

Using SwiftJava to wrap our Library on Android

1. Add a SwiftJava config file
2. Apply the SwiftJava build plugin
3. Build an AAR using Gradle and Swift Android SDK

1. Add swift-java config file

```
// Sources/Chia/swift-java.config

{
  "javaPackage": “net.frameo.chia",
  "mode": "jni",
  "memoryManagementMode": "allowGlobalAutomatic",
  "enableJavaCallbacks": true
}
```

2. Add JExtract build plugin

```
// Package.swift

let package = Package(
    name: "swift-chia",
    products: [
        .library(
            name: "Chia",
            targets: [“Chia"]
        )
    ],
    // ...
)
```

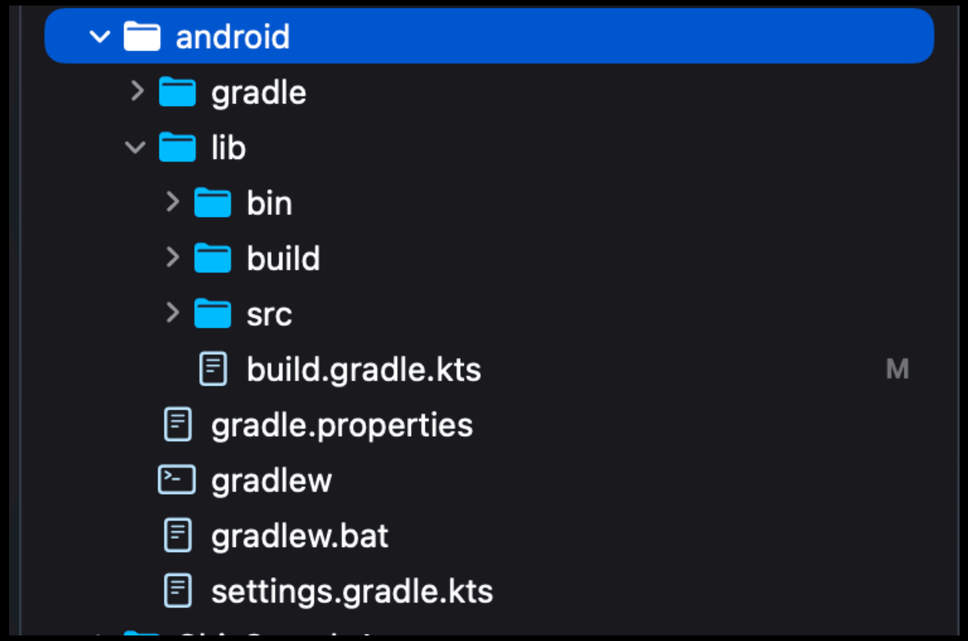
2. Add JExtract build plugin

```
// Package.swift

let isSwiftJavaBuild = ProcessInfo.processInfo.environment["SWIFT_JAVA_BUILD"] != nil

let package = Package(
    name: "swift-chia",
    products: [
        .library(
            name: "Chia",
            type: isSwiftJavaBuild ? .dynamic : .static,
            targets: [“Chia"]
        )
    ],
    // ...
)
```

```swift
// Package.swift

let isSwiftJavaBuild = ProcessInfo.processInfo.environment["SWIFT_JAVA_BUILD"] != nil

let package = Package(
    name: "swift-chia",
    products: [
        .library(
            name: "Chia",
            type: isSwiftJavaBuild ? .dynamic : .static,
            targets: ["Chia"]
        )
    ],
    // ...
)

if isSwiftJavaBuild {
    package.dependencies.append(
        .package(url: "https://github.com/swiftlang/swift-java", branch: "main"),
    )

    let chiaTarget = package.targets.first(where: { $0.name == "Chia" })!
    chiaTarget.dependencies.append(contentsOf: [
        .product(name: "SwiftJava", package: "swift-java"),
        .product(name: "CSwiftJavaJNI", package: "swift-java"),
        .product(name: "SwiftJavaRuntimeSupport", package: "swift-java"),
    ])

    chiaTarget.plugins = [
        .plugin(name: "JExtractSwiftPlugin", package: "swift-java")
    ]
}
```

3. Build Android Package

- Gradle project inside “android” subdirectory
- Based on “swift-android-examples” repo
- Calls swift build with Swift Android SDK
- Bundles .so and .java into Android package



📦 swift-chia.aar

```kotlin
// Main.kt

val chiaID = ChiaID.init()
val privateKeyBase64 = loadKey()
val client = ChiaClient.init(
    chiaID,
    privateKeyBase64,
    storage = TODO(),
    messageHandler = TODO()
)
```

```kotlin
// FileChiaStorage.kt

class FileChiaStorage(): ChiaStorage {
    override fun saveSession(session: ChiaSession) {}
    override fun deleteDevice(chiaID: ChiaID) {}
    override fun setActiveSession(chiaID: ChiaID, sessionID: String) {}
    // ...
}
```

```kotlin
// AndroidChiaMessageHandler.kt

class AndroidChiaMessageHandler(): ChiaMessageHandler {
    override fun messageReceived(message: ChiaMessage) {}
}
```

```java
// ChiaClient+Generated.java

public static <_T2 extends ChiaStorage, _T3 extends ChiaMessageHandler> ChiaClient init(
        ChiaID chiaID,
        java.lang.String privateKeyBase64,
        _T2 storage,
        _T3 messageHandler
) throws Exception
```

```kotlin
// Main.kt

val chiaID = ChiaID.init()
val privateKeyBase64 = loadKey()
val client = ChiaClient.init(
    chiaID,
    privateKeyBase64,
    storage = FileChiaStorage(),
    messageHandler = AndroidChiaMessageHandler()
)
```

```kotlin
// FileChiaStorage.kt

class FileChiaStorage(): ChiaStorage {
    override fun saveSession(session: ChiaSession) {}
    override fun deleteDevice(chiaID: ChiaID) {}
    override fun setActiveSession(chiaID: ChiaID, sessionID: String) {}
    // ...
}
```

```kotlin
// AndroidChiaMessageHandler.kt

class AndroidChiaMessageHandler(): ChiaMessageHandler {
    override fun messageReceived(message: ChiaMessage) {}
}
```

Swift Concurrency

```swift
// ChiaClient.swift

extension ChiaClient {

    /// Fetches messages from the server and notifies the message handler
    public func fetch() async throws {
        // ...
    }
}
```

```java
// ChiaClient.java

public java.util.concurrent.Future<java.lang.Void> fetch() {
    // ...
}
```

```swift
// SwiftChiaMessageHandler.swift

struct SwiftChiaMessageHandler: ChiaMessageHandler {
    let client: ChiaClient

    func messageReceived(_ message: ChiaMessage) {
        switch message.type {
        case .content(let contentMessage):
            Task {
                let outputFilePath = URL.temporaryDirectory.appending(path: "tmp")
                try await client.downloadAndDecryptMedia(
                    contentMessage.encryptedMedia,
                    streamingToPath: outputFilePath.path()
                )
            }

        default:
            return
        }
    }
}
```

```kotlin
// AndroidChiaMessageHandler.kt

class AndroidChiaMessageHandler(private val chiaClient: ChiaClient, private val context: Context): ChiaMessageHandler {
    override fun messageReceived(message: ChiaMessage) {
        when(val case = message.type.getCase(SwiftArena.ofAuto())) {
            is ChiaMessageType.Content -> {
                val contentMessage = case.arg0
                val outputFilePath = File.createTempFile("tmp", null, context.cacheDir).absolutePath
                chiaClient.downloadAndDecryptMedia(contentMessage.encryptedMedia, outputFilePath).get()
            }
            else -> return
        }
    }
}
```

Documentation

```swift
/// Requests a media upload URL for the given recipients.
///
/// This method contacts the media server to create a new media upload session
/// and returns the information required to upload the media, including the
/// upload URL and the associated media identifier.
///
/// - Parameter recipients: The recipients that the uploaded media will be shared with.
/// - Returns: A `MediaUpload` containing the upload URL and media identifier.
public func requestMediaUpload(
    recipients: [ChiaID]
) async throws -> MediaUpload {
```

```java
// ChiaClient.java

/**
 * Requests a media upload URL for the given recipients.
 *
 * <p>This method contacts the media server to create a new media upload session
 * and returns the information required to upload the media, including the
 * upload URL and the associated media identifier.
 *
 * <p>Downcall to Swift:
 * {@snippet lang=swift :
 * public func requestMediaUpload() async throws
 * }
 *
 * @param recipients The recipients that the uploaded media will be shared with.
 * @return A `MediaUpload` containing the upload URL and media identifier.
 */
public static java.util.concurrent.CompletableFuture<java.lang.Void> requestMediaUpload(ChiaID[] recipients)
```

SwiftJava

- Async support
- Nested types
- Arrays
- Implement protocols in Java
- CustomStringConvertible (toString)
- Javadocs support
- Improved UInt support

Wrap-up

- Low effort
- Saves time and frees up resources
- Utilize Swifts strengths
- Idiomatic API for Java developers